Dear Editors,

We submit "SafeMAP: Analysis-Guided Planning and Per-Function Validation for Safe C-to-Rust Translation" for consideration in the *Software Quality Journal*.

Context and importance. Memory-safety errors such as buffer overflows and use-after-free remain a leading cause of security vulnerabilities in C software, and migrating C code to safe Rust removes these whole classes of errors. Automatic translators, however, either keep C's pointer conventions in unsafe Rust or rely on large language models (LLMs) whose output is hard to check. A migration team therefore needs to know not only how much code was translated, but which translated functions can be trusted.

The work. The paper addresses this quality-assurance question. SafeMAP translates a C function only when static analysis finds a safe, testable Rust interface for it. It accepts a translation only if the code contains no `unsafe` and matches the original C function on generated inputs, with the C side run under AddressSanitizer and UndefinedBehaviorSanitizer so that inputs with undefined C behavior are excluded rather than used as expected outputs. Every other function is refused with a recorded reason.

Main results. We evaluated the frozen implementation under a sealed protocol on ten held-out C libraries (9,456 lines, 356 translation units):

- Across three runs, SafeMAP accepted 12-16 units under the sealed protocol. A declared amendment re-scored the same outputs with the lint check applied per function rather than per crate; SafeMAP then accepted 36-37 units (10%). With the change switched off, the re-scoring reproduces every sealed result, and the paper reports both sets of numbers.
- Under the amendment, a one-shot LLM given SafeMAP's planned interfaces accepted 28-31 units; the same model given only the C sources accepted 1-2. Raw C2Rust accepted none.
- Mutation analysis shows that the per-function check rejects 93% of non-equivalent compiling mutants of accepted functions.
- Most units are refused before translation, mainly because pointer ownership or aliasing cannot be resolved statically or because a C type has no safe Rust interface. The paper reports these reasons as a breakdown of what currently blocks safe automatic migration.

The results are modest and reported as such. Most of the gain over the plain LLM comes from the planned interfaces; SafeMAP's per-function loop adds 6-9 units over a one-shot LLM given the same interfaces, mostly in one library.

Fit with the Software Quality Journal. The paper is about assuring the quality of automatically produced code, which falls within the journal's interest in testing, verification, and tools for achieving software quality. It contributes (1) an acceptance procedure in which every decision is checked against the original program by sanitizer-guarded differential testing, (2) a measurement of that check's strength by mutation analysis, and (3) an empirical account, under a sealed protocol, of how far such checked migration reaches on production C code and what stops it.

All code, protocols, manifests, raw results, and table-generation scripts are public (https://doi.org/10.5281/zenodo.23619725). The manuscript is not under consideration elsewhere. All authors approved the submission.

Sincerely,
Jacob Sebastian Cyril (corresponding author), on behalf of the authors
Department of Computer Science, CHRIST (Deemed to be) University, Bangalore